DETAILED GUIDE

VM server setup for the multiplayer game: detailed guide

From an empty VirtualBox VM to a server that serves a Unity WebGL game over HTTPS and runs one Unity room process per match, with the evidence behind every choice.

This guide expands Steps 2-7 and 10 of the step-by-step tutorial (`docs/TUTORIAL.md`). It goes slower, explains why each choice was made, and says how much evidence stands behind each command.

> Every command and every choice in this guide is labelled with how much evidence stands behind it. Choices that came from a manual are marked **DOC**. Things that were really run in a sandbox are **TESTED**. Things that come from search summaries are **SUMMARY**. Everything else is **UNTESTED** and has to be proven on your own VM.

Repository: github.com/SynokJ/multiplayer_maze | Ubuntu Server 24.04 | nginx | Node.js 24 | certbot + DuckDNS

Contents

1. How to read the evidence labels
2. Part 1. The plan and the decisions
3. Part 2. Before you start
4. Part 3. Create the VM
5. Part 4. Log in from your PC with a key
6. Part 5. Base system: updates, clock, firewall
7. Part 6. A free name for the VM
8. Part 7. Install the software
9. Part 8. The HTTPS certificate
10. Part 9. Put the server application on the VM
11. Part 10. Prove it works, layer by layer
12. Part 11. Publishing builds and running the VM
13. Part 12. When it does not work
14. Appendix A. Changes made to the repository because of this verification
15. Appendix B. What is still unproven
16. Appendix C. Sources

How to read the evidence labels

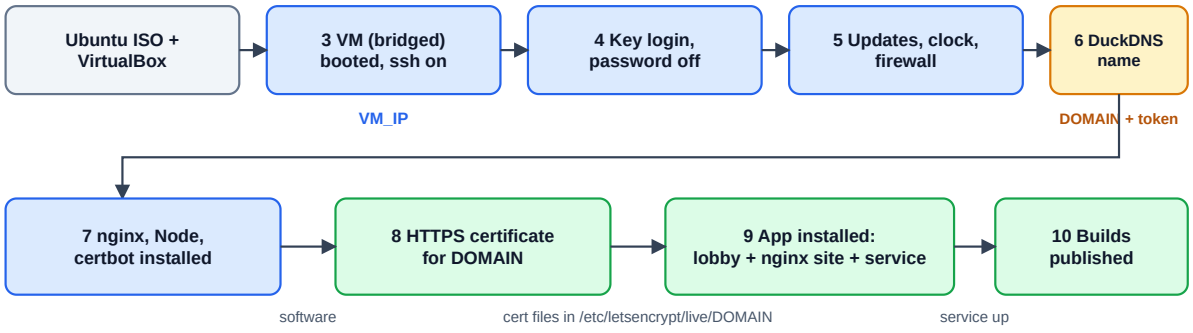
Everything in this guide carries one of four labels. They are deliberately strict.

Label	Meaning
DOC	The statement was read in primary documentation text during this work (nginx, systemd, OpenSSH, certbot, Let's Encrypt, Node.js release schedule, PyPI package page). The documents were read from their source repositories on GitHub and from the PyPI JSON API, because the public documentation websites were not reachable from the work environment.
TESTED	Executed in a sandbox: an Ubuntu 24.04 (noble) container with nginx 1.24.0 from the Ubuntu archive, Node.js 22.22.0 and 24.22.1, and the real files from this repository.
SUMMARY	Taken from web-search result summaries, not from a page that was read in full.
UNTESTED	Written from standard practice. Needs a real run on your VM.

What the sandbox could not do. It has no VirtualBox, no Ubuntu installer, no systemd running as PID 1, no browser and no Unity. It could not reach duckdns.org or letsencrypt.org, so a real certificate was never issued. Where this guide says TESTED for a service, the lobby and the room processes were run as the unprivileged user `maze` from the installed files behind a real nginx with TLS, and `systemctl` was replaced by a recording stub.

Part 1. The plan and the decisions

1.1 What you are building

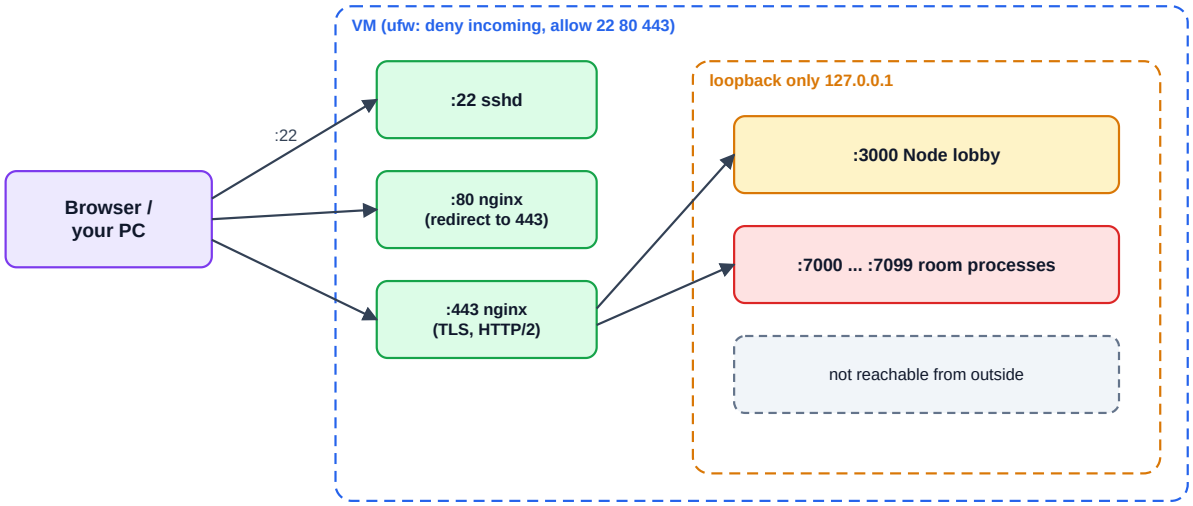


Step 6 (name) is needed before step 8, but not before step 7: you can install software while DNS settles.

The ten stages and what each hands to the next

Ten stages, each one needing something the previous stages produced. The ISO and VirtualBox give a VM; the VM gives an address; the address is given a free name; the name allows a certificate; the certificate allows the nginx site; the nginx site fronts the lobby; the builds from Unity go on top. If a stage fails, look at what it needed.

1.2 Which ports are open



Which ports are open and which stay on the VM

Only three ports are reachable from outside: 22 (ssh), 80 (redirects to https) and 443 (everything else). The lobby on 3000 and the rooms on 7000-7099 listen on 127.0.0.1 only. The nginx rule that forwards rooms accepts only paths `/r/7000` to `/r/7099`, so even a client that knows the lobby's port cannot talk to it directly.

1.3 Decision table

Decision	Choice	Why	Evidence
Linux release	**Ubuntu Server 24.04 LTS** (noble)	NodeSource lists Ubuntu 20.04, 22.04 and 24.04 as supported and does not list newer releases. The Ubuntu archive offers nginx 1.24.0.	**DOC** (NodeSource README). **TESTED**: `apt-cache policy nginx` on noble shows 1.24.0-2ubuntu7.x.
Web server	nginx from the Ubuntu archive	Regex `location` with a variable `proxy_pass` routes rooms without reloading.	**DOC**, **TESTED** (section 8.4)
Node.js	**24 LTS** from NodeSource, **22** as the fallback	Support ends 2028-04-30 for Node 24 and 2027-04-30 for Node 22. Neither is in "Active LTS" on 2026-10-27; both are in maintenance. Node 26 becomes LTS on 2026-10-28.	**DOC** (release schedule). Lobby **TESTED** on 22.22.0 and 24.22.1.
Do not use	Ubuntu's own `nodejs` package	It is Node 18 on 24.04, which is out of support.	**SUMMARY**
Certificate proof	**DNS-01**	Needs only a DNS record, no open ports, and works for servers that are not publicly reachable. HTTP-01 needs port 80 from the internet. It cannot validate IP addresses.	**DOC** (challenge types)
Free name	DuckDNS (`name.duckdns.org`)	Free; the certbot plugin exists and is maintained.	**DOC** (PyPI page)
Certbot install	Python virtual environment, not snap	The certbot documentation supports pip installs on a best-effort basis and only inside a virtual environment.	**DOC**
Rehearsal first	`--staging` certificate	Staging has much higher limits and issues untrusted test certificates.	**DOC**

Decision	Choice	Why	Evidence
Network mode	**Bridged** adapter	The VM gets its own LAN address; phones and other PCs can test. NAT needs per-port forwarding.	SUMMARY
Rooms	One Unity process per room on loopback	A WebGL client can never host.	NGO source, earlier research
Process supervision	systemd with `KillMode=control-group`	Stopping the lobby kills every room it started.	DOC (systemd.kill), TESTED emulation
Resource limits	`MemoryMax=1700M`, no `LimitNOFILE`	The systemd manual recommends `MemoryMax=` for memory and warns against raising the soft file limit casually.	DOC

1.4 Values you will create and reuse

Name	Example	Created in	Used in
`VM_IP`	`192.168.1.50`	3.4	4, 6, 9
`VM_USER`	`vmadmin`	3.3	4, 9
`DOMAIN`	`mymaze.duckdns.org`	6	8, 9, 10
`DUCK_TOKEN`	secret string	6	8 only
`TOKEN_SECRET`, `INTERNAL_KEY`	generated	9	stay on the VM

1.5 Sizing

Plan 2 CPU, 4 GB RAM and 20 GB disk. These are choices, not documented minimums. The lobby is limited to 1.7 GB so that the system keeps room; with 100-250 MB per room that is about 6-12 rooms, an estimate that has never been measured. Measure after the first real run (section 11.3).

Part 2. Before you start

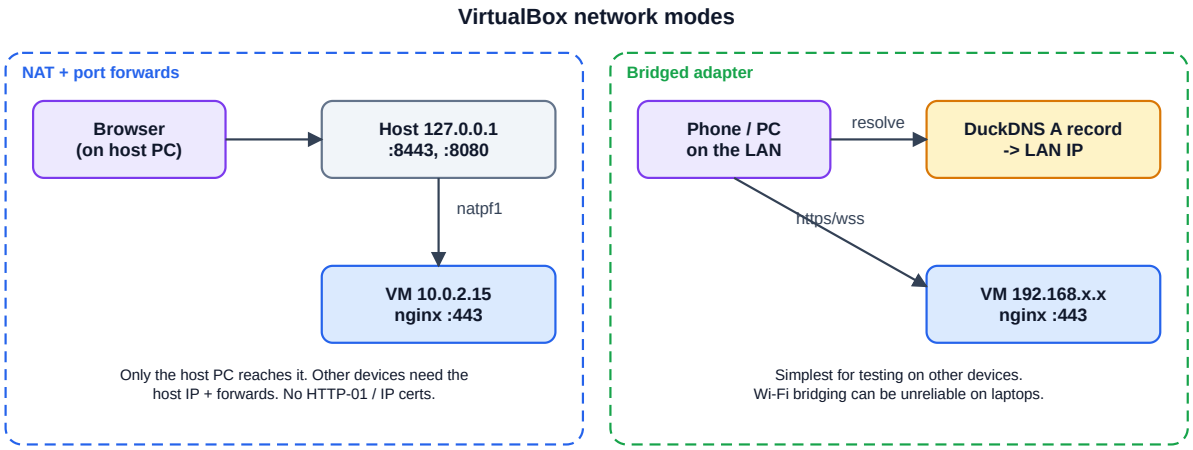
You need: a PC with an Intel or AMD processor (the Unity Linux server is an x86-64 program, and an ARM VM cannot run it; VirtualBox 7.2 can run ARM VMs on Apple-chip Macs, which does not help here); hardware virtualization enabled in the BIOS; VirtualBox 7.x from virtualbox.org (SUMMARY: current release line 7.2); the Ubuntu Server 24.04 LTS ISO from ubuntu.com; a free DuckDNS account; and about 30 minutes.

Check the ISO. Ubuntu publishes a `SHA256SUMS` file next to the ISO. On your PC:

```
sha256sum ubuntu-24.04*-live-server-amd64.iso      # Windows PowerShell: Get-FileHash <file> -
Algorithm SHA256
```

The value must match the one in `SHA256SUMS` (UNTESTED, standard practice).

Part 3. Create the VM



Bridged (recommended) or NAT

3.1 Create the machine in VirtualBox

Click **New** and fill in:

Field	Value	Note
Name	`maze-vm`	
ISO image	the Ubuntu Server ISO	
Skip Unattended Installation	**tick**	You answer the installer yourself; the unattended installer was not checked against 24.04.
Base memory	4096 MB	See 1.5
Processors	2	
Hard disk	20 GB, dynamically allocated	

3.2 Set the network adapter

Open **Settings, Network, Adapter 1**. Choose **Attached to: Bridged Adapter** and select the network card that connects your PC to the internet.

- **Bridged:** the VM gets its own address from your router. No port forwarding. Phones on the same Wi-Fi can reach it. Bridging over some Wi-Fi cards is unreliable (**SUMMARY** : driver limits); if the VM gets no address, choose another card or use NAT.
- **NAT with port forwards:** the VM is hidden behind the host. Each rule has the form `name,protocol,host IP,host port,guest IP,guest port`, the protocol is mandatory, and the VM must be powered off when you change rules with `modifyvm` (**SUMMARY**). Example for ssh and https:

```
VBoxManage modifyvm maze-vm --natpf1 "ssh,tcp,,2222,,22"
VBoxManage modifyvm maze-vm --natpf1 "https,tcp,,8443,,443"
```

With NAT you log in with `ssh -p 2222 vmadmin@127.0.0.1`, and in section 9 you must set `PUBLIC_PORT=8443`.

3.3 Install Ubuntu

Start the VM and answer the installer in this order. Screen names can differ slightly (`SUMMARY`).

Screen	Choose
Language, keyboard	your own
Type of install	Ubuntu Server (not "minimized")
Network	keep DHCP; note the address it shows
Proxy, mirror	leave the defaults
Storage	use the entire disk (defaults)
Profile	your name; server name `maze-vm` ; **username** `vmadmin` ; a strong password
Ubuntu Pro	skip
SSH	**tick "Install OpenSSH server"**. Optional: import your public key from GitHub.
Featured snaps	select nothing

Wait for "Install complete", choose **Reboot now**, and remove the ISO if the installer starts again (Settings, Storage).

3.4 First login and the VM's address

Log in on the VM window as `vmadmin` :

```
hostname -I          # first number = VM_IP, for example 192.168.1.50
```

Result: you know `VM_IP` . **Next:** you will not use the VM window again; section 4 moves you to your own terminal.

Part 4. Log in from your PC with a key

4.1 Make a key and install it

On your PC:

```
ssh-keygen -t ed25519 -C "maze-vm"              # press Enter for the default path; set a passphrase
```

Linux or macOS: `ssh-copy-id vmadmin@VM_IP` . Windows PowerShell (the `ssh` client is built in):

```
type $env:USERPROFILE\.ssh\id_ed25519.pub | ssh vmadmin@VM_IP "mkdir -p ~/.ssh && chmod 700 ~/.ssh &&
cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys"
```

Test: `ssh vmadmin@VM_IP` must log in without asking for the account password (a passphrase for the key is fine).

4.2 Turn off password login (only after 4.1 works)

Keep your first ssh window open until the end of this section.

```
sudo tee /etc/ssh/sshd_config.d/10-hardening.conf >/dev/null <<'EOF'
PasswordAuthentication no
PermitRootLogin no
PubkeyAuthentication yes
EOF
sudo sshd -t && sudo systemctl reload ssh
sudo sshd -T | grep -E '^(passwordauthentication|permitrootlogin|pubkeyauthentication) '
```

Expected: `passwordauthentication no`, `permitrootlogin no`, `pubkeyauthentication yes`. Open a **second** window and log in to prove the key still works before you close the first.

Why the file is named `10-`: OpenSSH uses the first value it finds for each keyword and reads included files in lexical order (**DOC**: sshd_config manual). The default for `PasswordAuthentication` is `yes` and the default for `PermitRootLogin` is `prohibit-password` (**DOC**). **TESTED**: with a competing `50-cloud-init.conf` that says `PasswordAuthentication yes`, `sshd -T` still reported `no`, because `10-hardening.conf` is read first. The service is named `ssh` on Ubuntu (**UNTESTED** here: no systemd).

Part 5. Base system: updates, clock, firewall

```
sudo apt update && sudo apt -y upgrade
[ -f /var/run/reboot-required ] && echo "reboot needed" || echo "no reboot needed"
timedatectl                                  # System clock synchronized: yes
```

- **Clock.** Certificates and join tokens depend on correct time. Ubuntu uses systemd-timesyncd by default (**SUMMARY**: Ubuntu server documentation). If `System clock synchronized` says `no`: `sudo timedatectl set-ntp true`.
- **Automatic security updates.** Ubuntu Server installs and enables `unattended-upgrades` by default (**SUMMARY**: Ubuntu server documentation). Check with `systemctl status unattended-upgrades --no-pager | head -3` and `cat /etc/apt/apt.conf.d/20auto-upgrades`.
- **Firewall.** Allow ssh **before** enabling the firewall, or you lock yourself out (**SUMMARY**). The `OpenSSH` profile exists because the openssh-server package registers it.

```
sudo apt install -y ufw
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw --force enable
sudo ufw status verbose
```

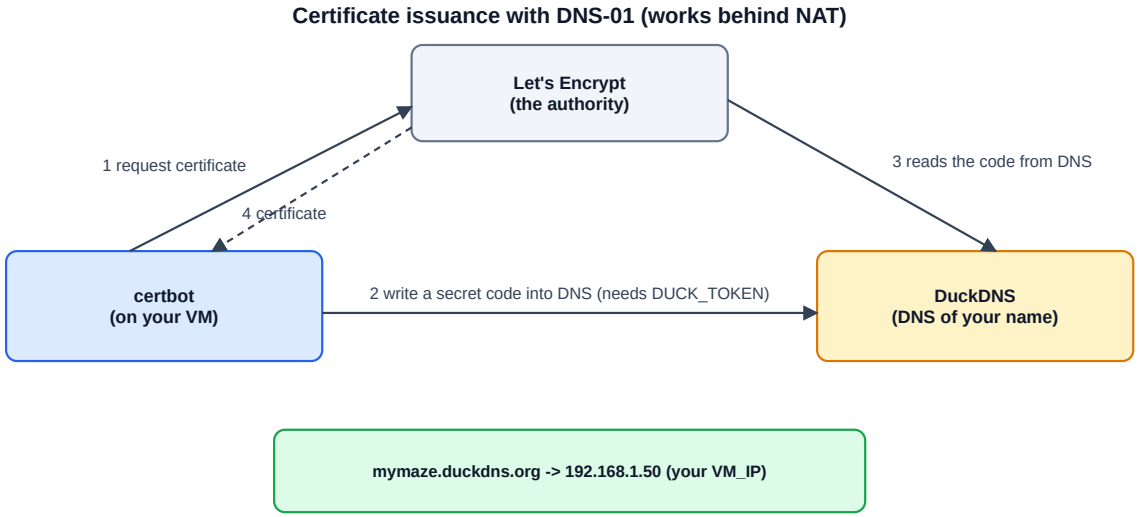
Result: a patched VM, correct clock, only ports 22, 80 and 443 open. All commands in this part are **UNTESTED** (the sandbox has no firewall support).

Part 6. A free name for the VM

6.1 Why a name, and why a private address is fine

Browsers trust a certificate only if it names the site they asked for, so you need a name. With the **DNS-01** challenge the certificate authority checks a TXT record in DNS and never connects to your server.

The documentation lists this as the challenge that "can validate domains whose web servers aren't publicly reachable", works without ports 80 or 443, and supports wildcards, and also lists its limits: it cannot validate IP addresses, and keeping DNS API credentials on the server is a risk (**DOC**: Let's Encrypt challenge types). That is why a name that points to a private `192.168.x.x` address still gets a real certificate.



How DNS-01 proves the name without opening your VM

6.2 Create the name

1. Sign in at duckdns.org (free).
2. In **sub domain** enter a name such as `mymaze` and press **add domain**. Your `DOMAIN` is `mymaze.duckdns.org`.
3. Set **current ip** to your `VM_IP` and press **update ip**.
4. Copy the **token** from the top of the page. It is your `DUCK_TOKEN`.

The token is an account secret: never put it in git or in the shell history. Section 8 enters it without echo.

Check (on your PC):

```
nslookup mymaze.duckdns.org                 # must print VM_IP
nslookup mymaze.duckdns.org 8.8.8.8         # same question to a public resolver
```

If the first one fails and the second works, your router blocks DNS answers that point to private addresses ("DNS rebind protection"; **UNTESTED**, a known router feature): add an exception or use another resolver. If the VM's address changes (DHCP), update DuckDNS again, or give the VM a fixed address in your router. DuckDNS's update interface takes `domains`, `token` and an optional `ip` (**SUMMARY**).

6.3 What the DuckDNS plugin does

The certbot plugin writes a TXT record at `_acme-challenge` for your name using your token, waits for it to propagate, and restores the old TXT record afterwards unless you pass `--dns-duckdns-no-txt-restore` (**DOC**: PyPI page, plugin 1.9.0). DuckDNS keeps one TXT record per name, so request each certificate name separately (**SUMMARY**).

Part 7. Install the software

```
sudo apt install -y nginx python3-venv curl openssl
apt-cache policy nginx | head -3            # Candidate: 1.24.0-2ubuntu7.x on Ubuntu 24.04
nginx -v
```

TESTED: on Ubuntu 24.04 the archive installs `nginx/1.24.0 (Ubuntu)`. The stock `/etc/nginx/nginx.conf` contains `gzip on;`. That matters later: Unity's `.gz` files are already compressed and must not be compressed again (section 9.4).

Node.js, using the method from the NodeSource README: download the script, read it, then run it. The README documents `setup_22.x`; the same pattern with `setup_24.x` is used by current guides (DOC for 22, SUMMARY for 24).

```
curl -fsSL https://deb.nodesource.com/setup_24.x -o nodesource_setup.sh
less nodesource_setup.sh                    # it runs as root and edits your apt configuration: read it first
sudo -E bash nodesource_setup.sh
sudo apt install -y nodejs
node -v && npm -v                           # v24.x
```

If `setup_24.x` is refused, use `setup_22.x`. The lobby declares `"engines": {"node": ">=22"}` and was TESTED on both 22.22.0 and 24.22.1: create room, WebSocket echo, join, and children killed on SIGTERM.

Part 8. The HTTPS certificate

8.1 Rules you are working inside

Limit	Value	Meaning for you
New certificates per registered domain	50 per 7 days	`duckdns.org` is on the Public Suffix List (checked in earlier research), so your `name.duckdns.org` counts as its own domain
Duplicate certificates (same names)	5 per 7 days, no override	Do not re-issue repeatedly while debugging
Failed validations	5 per identifier per account per hour	A wrong token burns attempts fast
Renewals	exempt from the first two limits; still count toward the last two	
Staging environment	same kinds of limits with higher values; certificates are **not trusted** by browsers	Use it to rehearse

DOC: Let's Encrypt rate limits and staging pages.

8.2 Install certbot and the DuckDNS plugin

```
sudo python3 -m venv /opt/certbot
sudo /opt/certbot/bin/pip install certbot certbot-dns-duckdns
sudo /opt/certbot/bin/certbot --version
sudo /opt/certbot/bin/certbot plugins | grep -i duckdns
```

DOC: pip installs are supported only in a virtual environment, on a best-effort basis. The plugin requires certbot 1.18 to below 6 and Python 3.10 or newer. **TESTED:** in the sandbox, `pip install certbot certbot-dns-duckdns` in a fresh virtual environment installed certbot 5.8.0 and certbot-dns-duckdns 1.9.0, and `certbot plugins` listed `dns-duckdns`. The plugin shows the options `--dns-duckdns-credentials`, `--dns-duckdns-token`, `--dns-duckdns-token-env`, `--dns-duckdns-propagation-seconds` and `--dns-duckdns-no-txt-restore`. The default propagation wait is 30 seconds; this guide uses 60.

8.3 Store the token safely

```
read -rsp "DuckDNS token: " DUCK_TOKEN; echo          # typed without echo
printf 'dns_duckdns_token=%s\n' "$DUCK_TOKEN" | sudo tee /root/duckdns.ini >/dev/null
sudo chmod 600 /root/duckdns.ini
unset DUCK_TOKEN
```

Token priority in the plugin: command line, then credentials file, then the `DUCKDNS_TOKEN` environment variable (**DOC**).

8.4 Rehearse with the staging server, then issue the real certificate

```
DOMAIN=mymaze.duckdns.org          # your DOMAIN
EMAIL=you@example.com

# 1) rehearsal: a separate certificate name keeps it apart from the real one
sudo /opt/certbot/bin/certbot certonly --non-interactive --agree-tos --email "$EMAIL" \
  --preferred-challenges dns --authenticator dns-duckdns \
  --dns-duckdns-credentials /root/duckdns.ini --dns-duckdns-propagation-seconds 60 \
  -d "$DOMAIN" --cert-name "$DOMAIN-staging" --staging

# 2) the real certificate
sudo /opt/certbot/bin/certbot certonly --non-interactive --agree-tos --email "$EMAIL" \
  --preferred-challenges dns --authenticator dns-duckdns \
  --dns-duckdns-credentials /root/duckdns.ini --dns-duckdns-propagation-seconds 60 \
  -d "$DOMAIN" --cert-name "$DOMAIN"

# 3) remove the rehearsal certificate
sudo /opt/certbot/bin/certbot delete --cert-name "$DOMAIN-staging"
```

How this follows the documentation: `--staging` (alias `--test-cert`) uses the staging server and issues untrusted test certificates; `--cert-name` is only a local label that decides the folder name; `certbot delete` removes all files for a certificate and needs `--cert-name` (**DOC**: certbot CLI reference). Certificates are stored in `/etc/letsencrypt/live/<cert-name>/`, and the documentation tells you to point your server there rather than copy the files (**DOC**). The explicit certificate name is why the real files end up exactly where the nginx file expects them.

Check:

```
sudo /opt/certbot/bin/certbot certificates
sudo ls /etc/letsencrypt/live/$DOMAIN/                                  # fullchain.pem
privkey.pem  ...
sudo openssl x509 -in /etc/letsencrypt/live/$DOMAIN/fullchain.pem -noout -issuer -subject -dates
```

The real certificate's issuer is Let's Encrypt and its validity is about 90 days. A rehearsal certificate shows a staging issuer. Issuance itself is **UNTESTED**: the sandbox could not reach Let's Encrypt or DuckDNS.

8.5 Renewal

Packaged installs configure renewal for you, but a virtual-environment install does not, so you add it (DOC: "Most Certbot installations come with automatic renewals preconfigured"). `certbot renew` renews only certificates that are due; `--force-renewal` exists to override that (DOC).

```
sudo mkdir -p /etc/letsencrypt/renewal-hooks/deploy
printf '#!/bin/sh\nsystemctl reload nginx\n' | sudo tee /etc/letsencrypt/renewal-hooks/deploy/reload-
nginx.sh >/dev/null
sudo chmod +x /etc/letsencrypt/renewal-hooks/deploy/reload-nginx.sh
echo '17 3 * * * root /opt/certbot/bin/certbot renew -q' | sudo tee /etc/cron.d/certbot-renew
>/dev/null
sudo /opt/certbot/bin/certbot renew --dry-run
```

Scripts in `renewal-hooks/deploy` run after each successful renewal, in alphabetical order (DOC). `--dry-run` runs the whole renewal against the staging server without saving anything, and does not run deploy hooks unless you add `--run-deploy-hooks` (DOC).

Result: certificate files for `DOMAIN` that nginx will use in section 9. **Next:** until this exists, the install script refuses to run.

Part 9. Put the server application on the VM

9.1 Layout

Path	Contents	Owner and mode
`/opt/maze/web/`	the Unity WebGL build	`root:www-data`, 755 / 644
`/opt/maze/server/`	the Unity Linux server build, and `fake-room.sh`	`maze:maze`, binary `+x`
`/opt/maze/lobby/`	the Node lobby and its `node_modules`	`maze:maze`
`/opt/maze/bin/publish.sh`	script that installs new builds	`root:root`, 755
`/etc/maze/lobby.env`	settings and secrets	`root:maze`, 640
`/etc/nginx/sites-available/maze.conf`	the nginx site, linked into `sites-enabled`	root
`/etc/systemd/system/maze-lobby.service`	the lobby service	root
`/run/maze-lobby/`	pid files of running rooms	created by systemd

9.2 Upload

On your PC, in the repository folder (see the tutorial, Step 1, to clone it):

```
VM=vmadmin@192.168.1.50            # VM_USER@VM_IP
ssh $VM "mkdir -p ~/upload"
scp -r server deploy $VM:~/upload/
```

On the VM, remove Windows line endings that git may have added, then run the installer:

```
find ~/upload -type f \( -name '*.sh' -o -name '*.service' -o -name '*.template' \) -exec sed -i
's/\r$//' {} +
sudo bash ~/upload/deploy/install-on-vm.sh mymaze.duckdns.org
```

9.3 What the installer does

TESTED in the sandbox (Ubuntu 24.04 container, root, `systemctl` stubbed; the template had its two IPv6 `listen` lines removed only because the sandbox has no IPv6): the script finished with exit code 0, created the `maze` account and folders, ran `npm install` as `maze`, wrote `/etc/maze/lobby.env` with permissions `root:maze 640` and secrets of 64 and 32 characters, installed the nginx site, passed `nginx -t`, and called `systemctl` in the order `reload nginx`, `daemon-reload`, `enable`, `restart`.

The installer, `deploy/install-on-vm.sh`:

```
#!/usr/bin/env bash
# Run ON THE VM, once, after the certificate exists:
#   sudo bash ~/upload/deploy/install-on-vm.sh <DOMAIN>
# Installs the lobby, nginx site and systemd service from the folders you uploaded (server/ and
deploy/).
set -euo pipefail
DOMAIN=${1:?usage: install-on-vm.sh <DOMAIN>   e.g. mymaze.duckdns.org}
HERE=$(cd "$(dirname "$0")" && pwd)
UP=$(dirname "$HERE")                                  # ~/upload
CERT=/etc/letsencrypt/live/$DOMAIN/fullchain.pem

[ "$(id -u)" -eq 0 ] || { echo "run with sudo"; exit 1; }
[ -f "$CERT" ] || { echo "No certificate at $CERT. Finish Step 5 first."; exit 1; }
command -v node  >/dev/null || { echo "node missing. Do Step 4 first."; exit 1; }
command -v nginx >/dev/null || { echo "nginx missing. Do Step 4 first."; exit 1; }

echo "== service account and folders"
id maze >/dev/null 2>&1 || useradd --system --home /opt/maze --shell /usr/sbin/nologin maze
mkdir -p /opt/maze/web /opt/maze/server /opt/maze/lobby /opt/maze/bin /etc/maze /var/www/acme
chown maze:maze /opt/maze

echo "== lobby code -> /opt/maze/lobby"
rm -rf /opt/maze/lobby/src /opt/maze/lobby/tools
cp -r "$UP/server/lobby/." /opt/maze/lobby/
cp "$UP/server/lobby/tools/fake-room.sh" /opt/maze/server/fake-room.sh
sed -i 's#"$(dirname "$0")/fake-room-server.mjs"#/opt/maze/lobby/tools/fake-room-server.mjs#'
/opt/maze/server/fake-room.sh
chmod +x /opt/maze/server/fake-room.sh
chown -R maze:maze /opt/maze/lobby /opt/maze/server
(cd /opt/maze/lobby && sudo -u maze env HOME=/opt/maze npm install --no-audit --no-fund)

echo "== secrets and settings -> /etc/maze/lobby.env (only created once)"
if [ ! -f /etc/maze/lobby.env ]; then
  {
    echo "LOBBY_PORT=3000"
    echo "SERVER_BIN=/opt/maze/server/fake-room.sh"
    echo "SERVER_CWD=/opt/maze/server"
    echo "PORT_MIN=7000"
    echo "PORT_MAX=7099"
    echo "MAX_ROOMS=8"
    echo "PUBLIC_HOST=$DOMAIN"
    echo "PUBLIC_PORT=443"
    echo "PUBLIC_TLS=true"
    echo "TOKEN_SECRET=$(openssl rand -hex 32)"
    echo "INTERNAL_KEY=$(openssl rand -hex 16)"
  } > /etc/maze/lobby.env
  chown root:maze /etc/maze/lobby.env
  chmod 640 /etc/maze/lobby.env
fi

echo "== nginx site"
sed "s/__DOMAIN__/$DOMAIN/g" "$UP/deploy/nginx-maze.conf.template" > /etc/nginx/sites-
available/maze.conf
ln -sf /etc/nginx/sites-available/maze.conf /etc/nginx/sites-enabled/maze.conf
rm -f /etc/nginx/sites-enabled/default
```

```
nginx -t
systemctl reload nginx

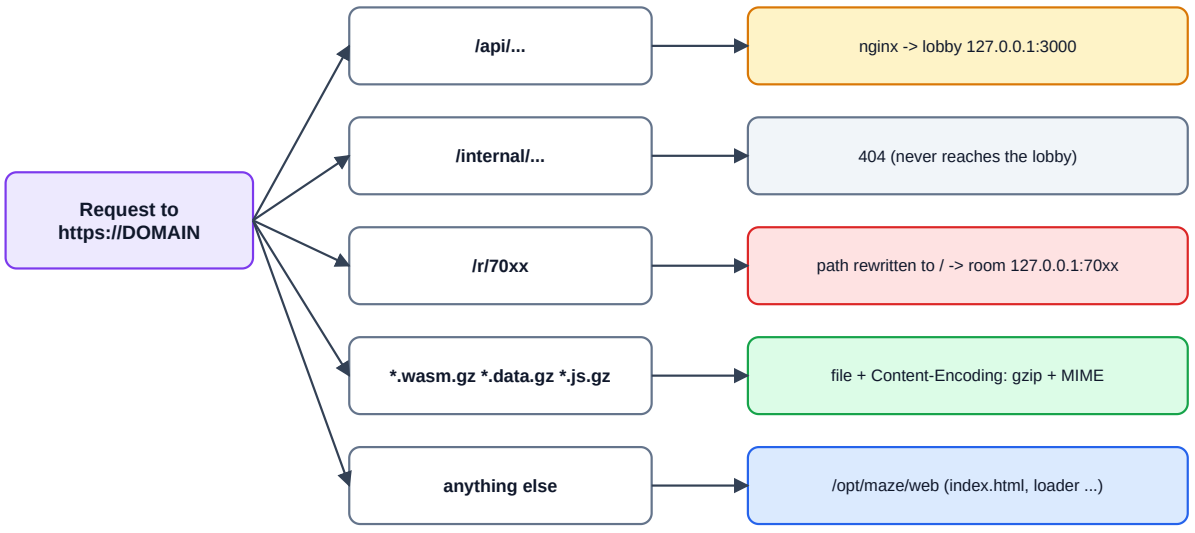
echo "== systemd service"
cp "$UP/deploy/maze-lobby.service" /etc/systemd/system/maze-lobby.service
install -m 755 "$UP/deploy/publish-on-vm.sh" /opt/maze/bin/publish.sh
systemctl daemon-reload
systemctl enable maze-lobby
systemctl restart maze-lobby

echo
echo "Done. Lobby on 127.0.0.1:3000, nginx on 443 for $DOMAIN."
echo "Check:  curl -s https://$DOMAIN/api/health"
```

Step in the script	Why
Refuses to run without the certificate, `node` or `nginx`	Makes the dependency on parts 7 and 8 explicit
`useradd --system ... maze`	Lobby and rooms never run as an administrator
Copies `server/lobby` and runs `npm install`	`ws` is a dev dependency used by the stand-in room, so a plain `npm install` is intended
Rewrites `fake-room.sh` to an absolute path	The stand-in must work from any working directory
Creates `lobby.env` once	Re-running the installer keeps your secrets
`sed` fills `__DOMAIN__` into the nginx template	One template for every name
`nginx -t` before reload	A broken config never replaces a working one silently

The lobby starts `fake-room.sh` (a tiny WebSocket echo server that takes the same arguments as the Unity binary) until you publish the real Unity server. This lets you test everything on the server before Unity is involved.

9.4 The nginx site, explained



Rooms that do not exist answer 502; paths outside 7000-7099 fall through to 404.

Where nginx sends each kind of request

The file in the repository. `__DOMAIN__` is replaced by the installer.

```
# deploy/nginx-maze.conf.template
# nginx site for the maze server. deploy/install-on-vm.sh fills in __DOMAIN__ and installs it as
# /etc/nginx/sites-available/maze.conf (linked into sites-enabled/). Checked with nginx 1.24.0 (Ubuntu
24.04).
# The first directives (log_format, map, limit_*_zone) belong to the http{} context, which includes
sites-enabled/.
log_format rooms '$remote_addr [$time_local] "$request_method $request_uri" $status
upstream=$upstream_addr up_status=$upstream_status';
map $http_upgrade $connection_upgrade {
    default upgrade;
    ''      close;
}
limit_conn_zone $binary_remote_addr zone=perip:10m;
limit_req_zone  $binary_remote_addr zone=api:10m rate=10r/s;

server {
    listen 80;
    listen [::]:80;
    server_name __DOMAIN__;
    location /.well-known/acme-challenge/ { root /var/www/acme; }   # only used by HTTP-01 on a public
host
    location / { return 301 https://$host$request_uri; }
}

server {
    listen 443 ssl http2;          # works on every nginx version (newer ones print a harmless
'deprecated' notice)
    listen [::]:443 ssl http2;
    server_name __DOMAIN__;

    ssl_certificate     /etc/letsencrypt/live/__DOMAIN__/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/__DOMAIN__/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_session_cache shared:SSL:10m;

    server_tokens off;
    limit_conn perip 40;
    gzip off;                                          # ADDED: never re-compress Unity's pre-
compressed files
    add_header X-Content-Type-Options nosniff always;   # NOTE: any add_header inside a location
REPLACES inherited ones

    # --- Unity WebGL build (static) ---
    root /opt/maze/web;
    index index.html;

    location / {
        try_files $uri $uri/ =404;        # "/" resolves to index.html through the index directive
        add_header Cache-Control "no-cache";
        add_header X-Content-Type-Options nosniff always;   # repeated because this location sets its
own add_header
    }

    # Pre-compressed Unity files: correct Content-Encoding + real MIME type
    location ~ \.wasm\.br$   { types {} default_type application/wasm;         add_header Content-
Encoding br;   add_header Cache-Control "public, max-age=31536000, immutable"; }
    location ~ \.js\.br$     { types {} default_type application/javascript;   add_header Content-
Encoding br;   add_header Cache-Control "public, max-age=31536000, immutable"; }
    location ~ \.data\.br$   { types {} default_type application/octet-stream; add_header Content-
Encoding br;   add_header Cache-Control "public, max-age=31536000, immutable"; }
    location ~ \.wasm\.gz$   { types {} default_type application/wasm;         add_header Content-
Encoding gzip; add_header Cache-Control "public, max-age=31536000, immutable"; }
    location ~ \.js\.gz$     { types {} default_type application/javascript;   add_header Content-
Encoding gzip; add_header Cache-Control "public, max-age=31536000, immutable"; }
    location ~ \.data\.gz$   { types {} default_type application/octet-stream; add_header Content-
Encoding gzip; add_header Cache-Control "public, max-age=31536000, immutable"; }
    location ~ \.wasm$       { types {} default_type application/wasm; }

    # --- Lobby REST API ---
    location /api/ {
        limit_req zone=api burst=20 nodelay;
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $remote_addr;      # overwrite, never append a client-
supplied value
```

```
        proxy_set_header X-Forwarded-Proto $scheme;
        client_max_body_size 4k;
    }
    location /internal/ { return 404; }                     # heartbeat endpoint is loopback only

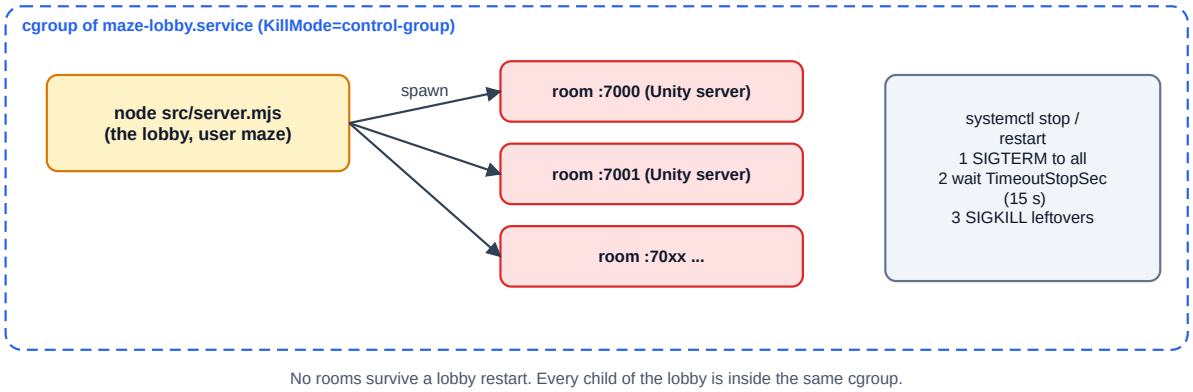
    # --- Game rooms: wss://HOST/r/7000 .. /r/7099 -> ws://127.0.0.1:<port>/ ---
    location ~ ^/r/(?<room_port>70[0-9][0-9])$ {
        proxy_pass http://127.0.0.1:$room_port/;            # variable + URI: the URI "/" replaces
/r/70xx; IP literal needs no resolver
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection $connection_upgrade;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $remote_addr;
        proxy_read_timeout 3600s;                           # default 60 s would cut idle sockets
        proxy_send_timeout 3600s;
        proxy_buffering off;
        limit_conn perip 12;
        access_log /var/log/nginx/rooms.log rooms;
        client_max_body_size 0;
    }
}
```

Each important line and its basis:

1. **`proxy_http_version 1.1` plus `Upgrade` and `Connection` headers.** The nginx WebSocket page shows exactly these. Its example comments the version line with "before version 1.29.7", and the proxy module says the default became 1.1 only in 1.29.7 and was 1.0 before. Ubuntu 24.04 has 1.24.0, so the line is required (**DOC**). The `map` that sends `Connection: close` when no upgrade was requested is a common pattern; the plain documented form is `Connection "upgrade"` (**UNTESTED** which is better).
2. **`proxy_pass http://127.0.0.1:$room_port/;`.** The proxy module says that when a URI is given in the directive "it is passed to the server as is, replacing the original request URI" and that variables are allowed in the address (**DOC**). So `/r/7003` reaches the room as `/`, which is the only path a Unity server accepts. **TESTED:** a stand-in room that accepts only `/` answered a secure WebSocket opened on `/r/7000` through nginx and echoed a binary frame `[1, 2, 3, 4]`.
3. **`proxy_read_timeout 3600s`.** The default is 60 s, applies between two successive reads, and a WebSocket is closed if the server sends nothing for that long; server-sent ping frames reset it (**DOC**). Rooms are idle between moves, so the value is raised.
4. **`gzip off`.** The stock `nginx.conf` has `gzip on;` (**TESTED**). Compressing Unity's `.gz`/`.br` files a second time produces the browser error "Unable to parse Build/...".
5. **Per-extension locations.** They set `Content-Type` (`application/wasm` for `.wasm.gz`) and `Content-Encoding: gzip`. **TESTED** responses: `.wasm.gz` gives `application/wasm` and `gzip`; `.data.gz` gives `application/octet-stream`; `.framework.js.gz` gives `application/javascript`; all with `cache-control: public, max-age=31536000, immutable`.
6. **`try_files $uri $uri/ =404`.** The first version of this file had `try_files $uri =404`, and **testing found** that `https://DOMAIN/` then returned 404 because `/` is a directory. The fixed line returns `index.html` (200, `cache-control: no-cache`).
7. **`listen 443 ssl http2`.** The separate `http2 on;` directive appeared in nginx 1.25.1 (**DOC**), so it is not available on 1.24.0; the parameter form works on both. Newer nginx versions print a deprecation notice but run. **TESTED:** `nginx -t` passes on 1.24.0 and curl negotiated HTTP/2 for the page.
8. **IPv6.** The file listens on `[::]` as well. **TESTED** failure in a container without IPv6: `socket() [::]:80 failed (97: Address family not supported by protocol)`. A normal VirtualBox VM has IPv6; if yours does not, delete the two `listen [::]` lines.

9. `/internal/` **returns 404.** The room processes report to the lobby on loopback; the public site must not expose that. **TESTED:** `POST /internal/rooms/AAAAA/heartbeat` through nginx gives 404.
10. **Rooms outside the range and dead rooms. TESTED:** `/r/9999` gives 404 (no location matches), and `/r/7050` with no process gives 502.

9.5 The service, explained



The lobby and its rooms live and die together

The file in the repository.

```
# deploy/maze-lobby.service
# /etc/systemd/system/maze-lobby.service   (installed by deploy/install-on-vm.sh; `systemd-analyze
verify` passes)
[Unit]
Description=Maze lobby and room supervisor
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=maze
Group=maze
WorkingDirectory=/opt/maze/lobby
EnvironmentFile=/etc/maze/lobby.env
Environment=NODE_ENV=production
ExecStart=/usr/bin/node src/server.mjs
Restart=on-failure
RestartSec=2

# children (Unity servers) live in this unit's cgroup and die with it
KillMode=control-group
KillSignal=SIGTERM
TimeoutStopSec=15
RuntimeDirectory=maze-lobby
RuntimeDirectoryMode=0750

TasksMax=512
MemoryMax=1700M
MemoryHigh=1500M

NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=true
PrivateTmp=true
PrivateDevices=true
ProtectKernelTunables=true
ProtectKernelModules=true
ProtectControlGroups=true
RestrictAddressFamilies=AF_INET AF_INET6 AF_UNIX
RestrictNamespaces=true
LockPersonality=true
SystemCallArchitectures=native
```

```
ReadWritePaths=/opt/maze/server

StandardOutput=journal
StandardError=journal
SyslogIdentifier=maze-lobby

[Install]
WantedBy=multi-user.target
```

Setting	Meaning	Evidence
`KillMode=control-group`	When the unit stops, all remaining processes in its cgroup are killed. Rooms are children of the lobby, so they die with it.	**DOC**
`KillSignal=SIGTERM`, `TimeoutStopSec=15`	SIGTERM first; after the timeout the termination is repeated and SIGKILL is sent.	**DOC**
`NoNewPrivileges=true`	The service and its children can never gain privileges through `execve()`.	**DOC**
`ProtectSystem=strict` + `ReadWritePaths=/opt/maze/server`	The whole file system is read-only except the API file systems; `ReadWritePaths=` allows exceptions.	**DOC**
`RuntimeDirectory=maze-lobby`	`/run/maze-lobby` is created when the unit starts and removed when it stops (pid files live there).	**DOC**. Whether it stays writable under `ProtectSystem=strict` was not shown in the text read; the same page says related directory settings are excluded from `ProtectSystem=`. If the lobby logs a read-only error, add `ReadWritePaths=/run/maze-lobby`.
`MemoryMax=1700M`, `MemoryHigh=1500M`	Memory ceiling; the manual recommends `MemoryMax=` for this.	**DOC**
`LimitNOFILE` (removed)	An earlier draft set `LimitNOFILE=8192`. The manual marks the setting "do not use" and warns about raising the soft limit above 1024 because of `select()`; the hard limit already defaults to a high value.	**DOC**; changed after reading

TESTED: `systemd-analyze verify deploy/maze-lobby.service` printed no errors (every directive is known and parses). The unit has not been started by a real systemd.

9.6 The settings file

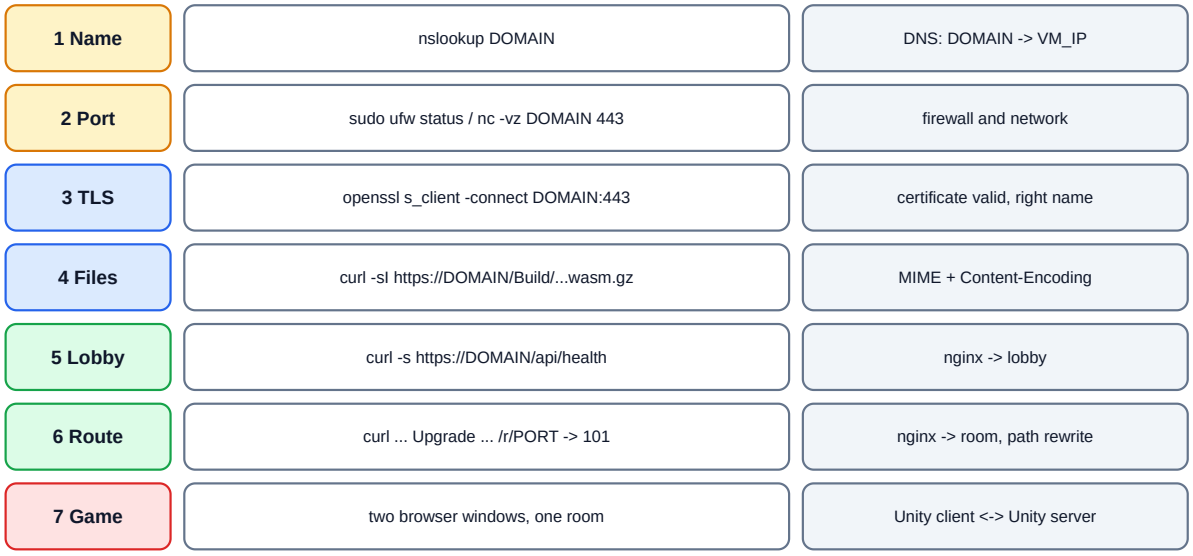
The documentation file in the repository. The installer creates the real `/etc/maze/lobby.env` with generated secrets.

```
# deploy/lobby.env.example
# /etc/maze/lobby.env   (chmod 640, owner root:maze)
LOBBY_PORT=3000
SERVER_BIN=/opt/maze/server/MazeServer.x86_64
SERVER_CWD=/opt/maze/server
PORT_MIN=7000
PORT_MAX=7099
MAX_ROOMS=8
```

```
PUBLIC_HOST=maze.duckdns.org
PUBLIC_PORT=443
PUBLIC_TLS=true
TOKEN_SECRET=<openssl rand -hex 32>
INTERNAL_KEY=<openssl rand -hex 16>
```

Variable	Default	Meaning
`LOBBY_HOST`, `LOBBY_PORT`	`127.0.0.1`, `3000`	Where the lobby listens. Keep the host on loopback.
`SERVER_BIN`, `SERVER_CWD`	the Unity binary, its folder	Program started for every room. The installer sets it to the stand-in `fake-room.sh`; `publish.sh` switches it to `MazeServer.x86_64`.
`PORT_MIN`, `PORT_MAX`	`7000`, `7099`	Room ports. Must match the regex in the nginx file (`70[0-9][0-9]`).
`MAX_ROOMS`, `MAX_PLAYERS`	`8`, `8`	Limits. Set `MAX_ROOMS` from measured memory.
`TOKEN_SECRET`	none, at least 32 characters	Signs join tokens. Passed to rooms through the environment, never the command line.
`INTERNAL_KEY`	none, at least 16 characters	Rooms send it when reporting to the lobby.
`TOKEN_TTL_SEC`	`60`	How long a join token is valid.
`READY_TIMEOUT_MS`	`30000`	A room that is not listening by then is killed.
`HEARTBEAT_TIMEOUT_MS`	`30000`	A room that stops reporting is killed.
`EMPTY_GRACE_MS`	`60000`	An empty room is closed after this.
`PUBLIC_HOST`, `PUBLIC_PORT`, `PUBLIC_TLS`	host name, `443`, `true`	What the lobby tells browsers to connect to. With NAT use port `8443`.
`ROOM_PATH_PREFIX`	`/r/`	Must match the nginx location.
`RUN_DIR`	`/run/maze-lobby`	Pid files.
`ALLOWED_ORIGIN`	empty	Empty means same-origin only.

Part 10. Prove it works, layer by layer



Test from the top. A failure at a lower number hides everything above it.

The verification ladder

Test in this order. If a layer fails, stop and fix it; a failure at a lower number hides everything above it. Replace `DOMAIN` with your name. Layers 5-6 use only the stand-in room, so Unity is not needed yet.

```
nslookup DOMAIN                                                    # 1 name -> VM_IP
sudo ufw status | grep -E '22|80|443'                              # 2 firewall
echo | openssl s_client -connect DOMAIN:443 -servername DOMAIN 2>/dev/null | openssl x509 -noout -
issuer -dates   # 3 TLS
curl -sI https://DOMAIN/ | head -1                                 # 4 files (200 after
publishing a build)
curl -s https://DOMAIN/api/health                                  # 5 lobby:
{"ok":true,"rooms":0}
curl -s -X POST https://DOMAIN/api/rooms -H 'content-type: application/json' -d '{"playerName":"Ann"}'
# 5 shows "path":"/r/7000"
curl -i -N --max-time 3 --http1.1 -H "Connection: Upgrade" -H "Upgrade: websocket" \
  -H "Sec-WebSocket-Version: 13" -H "Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ=="
https://DOMAIN/r/7000   # 6 expect 101
sudo tail -n 3 /var/log/nginx/rooms.log                            # 6 "GET /r/7000" 101
upstream=127.0.0.1:7000
```

Two details found by testing:

- The `Sec-WebSocket-Key` must be 16 bytes in base64. **TESTED:** the older example value `SGVsbG8sIHdvcmxkIQ==` (13 bytes) made the room answer `400`, and `rooms.log` showed `up_status=400`. The value above is the example from the WebSocket specification and gives `101`.
- `curl` stops after `--max-time` seconds because a WebSocket never closes by itself.

10.1 What the sandbox actually printed (TESTED)

```
GET  /api/health                      -> {"ok":true,"rooms":0}
GET  /Build/test.wasm.gz   -> HTTP/2 200, content-type: application/wasm, content-encoding: gzip,
cache-control: public, max-age=31536000, immutable
GET  /                     -> HTTP/2 200, content-type: text/html, cache-control: no-cache      (after
the try_files fix)
POST /internal/rooms/AAAAA/heartbeat  -> 404
GET  http://DOMAIN/                   -> 301 Location: https://DOMAIN/
POST /api/rooms  {"playerName":"Ann"} -> {"room":{"id":"WU6R8",
...},"url":"wss://maze.test:443/r/7000","path":"/r/7000", ...}
```

```
WebSocket /r/7000 (bad key)           -> 400   rooms.log: "GET /r/7000" 400 upstream=127.0.0.1:7000
up_status=400
WebSocket /r/7000 (good key)          -> HTTP/1.1 101 Switching Protocols
                                         rooms.log: "GET /r/7000" 101 upstream=127.0.0.1:7000
up_status=101
wss binary echo through nginx         -> [1,2,3,4] returned
WebSocket /r/9999                     -> 404       WebSocket /r/7050 (no process) -> 502
POST /api/rooms {"playerName":"<script>"} -> 400   POST /api/rooms/ZZZZZ/join -> 404
```

Then, with the lobby running as the unprivileged user `maze`: `ps` showed both the lobby and the room process owned by `maze`, a pid file `5SWWW.pid` appeared in `/run/maze-lobby/`, a second player could join, and sending SIGTERM to the lobby left no `maze` process running.

Part 11. Publishing builds and running the VM

11.1 Publish the Unity builds

Upload the two build folders and run the publish script (tutorial Step 10 shows the full commands):

```
# on your PC
scp -r "multiplayer game/Builds/WebGL/0.1.0"       $VM:~/upload/build-web
scp -r "multiplayer game/Builds/LinuxServer/0.1.0" $VM:~/upload/build-server
# on the VM
sudo /opt/maze/bin/publish.sh
```

The publish script, `deploy/publish-on-vm.sh`:

```
#!/usr/bin/env bash
# Run ON THE VM after uploading builds:   sudo /opt/maze/bin/publish.sh
# Expects:  ~/upload/build-web/ (WebGL build)   and/or   ~/upload/build-server/ (Linux server build).
# Only the folders you uploaded are replaced.
set -euo pipefail
[ "$(id -u)" -eq 0 ] || { echo "run with sudo"; exit 1; }
UPLOAD=${UPLOAD_DIR:-$(getent passwd "${SUDO_USER:-root}" | cut -d: -f6)/upload}

if [ -d "$UPLOAD/build-web" ]; then
  echo "== WebGL build -> /opt/maze/web"
  rm -rf /opt/maze/web/*
  cp -r "$UPLOAD/build-web/." /opt/maze/web/
  chown -R root:www-data /opt/maze/web
  find /opt/maze/web -type d -exec chmod 755 {} +
  find /opt/maze/web -type f -exec chmod 644 {} +
fi

if [ -d "$UPLOAD/build-server" ]; then
  echo "== Linux server build -> /opt/maze/server"
  find /opt/maze/server -mindepth 1 ! -name fake-room.sh -delete
  cp -r "$UPLOAD/build-server/." /opt/maze/server/
  chmod +x /opt/maze/server/MazeServer.x86_64
  chown -R maze:maze /opt/maze/server
  # switch the lobby from the stand-in to the real Unity server
  sed -i 's#^SERVER_BIN=.*#SERVER_BIN=/opt/maze/server/MazeServer.x86_64#' /etc/maze/lobby.env
fi

nginx -t && systemctl reload nginx
systemctl restart maze-lobby        # running rooms end; fine while developing
echo "Published. Open https://$(grep '^PUBLIC_HOST=' /etc/maze/lobby.env | cut -d= -f2)/"
```

TESTED with stand-in builds: the WebGL folder was copied to `/opt/maze/web` with mode `root:www-data 644`; the server folder was copied to `/opt/maze/server`, made executable and owned by `maze`; `fake-room.sh` survived; `SERVER_BIN` in `lobby.env` switched to `MazeServer.x86_64`; after the lobby restarted, a

new room was started through that path and the new page and `.wasm.gz` headers were served. Only the folders you upload are replaced.

11.2 Daily operations

Task	Command
Lobby and room messages	`sudo journalctl -u maze-lobby -f`
Every room connection	`sudo tail -f /var/log/nginx/rooms.log`
nginx errors	`sudo tail -f /var/log/nginx/error.log`
Restart the lobby (ends all rooms)	`sudo systemctl restart maze-lobby`
Test and reload nginx	`sudo nginx -t && sudo systemctl reload nginx`
Certificate status	`sudo /opt/certbot/bin/certbot certificates`
Reboot test	`sudo reboot`, then repeat layers 5-6 of part 10
Limit log size	a file `/etc/systemd/journald.conf.d/maze.conf` with `[Journal]`, `SystemMaxUse=200M`, `MaxRetentionSec=1week`, then `sudo systemctl restart systemd-journald` (**UNTESTED**)

11.3 Measure before you trust the room limit

After two players are in a room: `ps -o rss=,args= -u maze | grep MazeServer` shows memory per room in kilobytes. Multiply by the number of rooms you want, add about 400 MB for the system, nginx and the lobby, and set `MAX_ROOMS` below that. This is the measurement that replaces the estimate in 1.5.

11.4 Back up

Keep copies of `/etc/maze/lobby.env` (changing it invalidates nothing, but losing it means new secrets), `/etc/letsencrypt/`, `/root/duckdns.ini` and your ssh keys. The builds and the lobby come from your repository.

11.5 Moving to Node.js 26

Node 22 maintenance ends 2027-04-30. To change version on the VM, run the NodeSource script for the new version, `sudo apt install -y nodejs`, `node -v`, then `sudo systemctl restart maze-lobby`. Repeat the lobby test (layers 5-6) afterwards.

Part 12. When it does not work

Symptom	Likely cause	What to do
`nginx -t`: `socket() [::]:80 failed (97 ...)`	The system has no IPv6	Delete the two `listen [::]` lines (9.4, item 8)

Symptom	Likely cause	What to do
`nginx -t`: `unknown directive "http2"`	A newer or older nginx than the file expects	This repo's file uses `listen 443 ssl http2`, which works on 1.24 and later; on a custom nginx keep the same form
`https://DOMAIN/` gives 404	`try_files` without `$uri/` (old copy of the file)	Use the file from the repository (9.4, item 6)
Browser: "Unable to parse Build/....gz"	Wrong `Content-Encoding`, or nginx compressed twice	`curl -sI` the file; keep `gzip off`; see 9.4 items 4-5
WebSocket test gives `400`	`Sec-WebSocket-Key` is not 16 bytes, or a header is missing	Copy the command in part 10 exactly
WebSocket test gives `502`	No room process on that port	`sudo journalctl -u maze-lobby -n 30`; create a room first
WebSocket test gives `404`	Port outside 7000-7099, or typo in the path	Use the port from the lobby answer
Lobby exits at start: "TOKEN_SECRET (>=32 chars)..."	`/etc/maze/lobby.env` missing or short	Re-run the installer, or fill the file
`room_failed_to_start` (500)	Binary missing, not executable, or does not listen within 30 s	Run it by hand as `maze`: `sudo -u maze /opt/maze/server/MazeServer.x86_64 -batchmode -nographics -port 7099 -roomId TEST1 -logFile -`
Lobby logs a read-only file system error for `/run/maze-lobby`	`ProtectSystem=strict`	Add `ReadWritePaths=/run/maze-lobby` to the unit (9.5)
certbot: "Timed out waiting for DNS"	Propagation too short or wrong token	Raise `--dns-duckdns-propagation-seconds` to 120; check `/root/duckdns.ini`
certbot: rate limit errors	Repeated failures	Wait; rehearse with `--staging` first (8.1)
nginx still shows the old certificate after renewal	No reload hook	Create the deploy hook (8.5)
Phone cannot open the site, PC can	Router blocks private DNS answers, or the phone is on another network	Section 6.2
Cannot log in after hardening	Key not installed before password login was disabled	Use the VirtualBox console to restore password login, or fix `authorized_keys`
VM address changed	DHCP renewal	Update DuckDNS; reserve the address in the router

Appendix A. Changes made to the repository because of this verification

File	Change	Found by
`deploy/nginx-maze.conf.template`	`try_files $uri =404` became `try_files $uri $uri/ =404`	Sandbox test: `/` returned 404

File	Change	Found by
`deploy/maze-lobby.service`	Removed `LimitNOFILE=8192`	systemd manual says "do not use" and explains the `select()` limit
`docs/TUTORIAL.md` and PDF, Step 7	WebSocket key replaced with a valid 16-byte value	Sandbox test: old key gave 400
`docs/TUTORIAL.md` and PDF, Step 4	Node install uses the documented download-then-run method, Node 24	NodeSource README, Node release schedule
`docs/TUTORIAL.md` and PDF, Step 5	Staging rehearsal uses `--cert-name NAME-staging`, then `certbot delete`; no `--force-renewal`	certbot CLI reference

Appendix B. What is still unproven

Real certificate issuance with DuckDNS; the VirtualBox and Ubuntu installer screens; `ufw`, `fail2ban` and `unattended-upgrades` behaviour; the NodeSource `setup_24.x` script name (only `setup_22.x` is in the README that was read); the lobby under a real systemd (including `RuntimeDirectory` with `ProtectSystem=strict`); browsers connecting to `wss://` with a real certificate; and the Unity server and WebGL client. The last item is tutorial Step 11.

Appendix C. Sources

- nginx WebSocket proxying: https://nginx.org/en/docs/http/websocket.html
- nginx proxy module (`proxy_pass`, `proxy_read_timeout`, `proxy_http_version`): https://nginx.org/en/docs/http/ngx_http_proxy_module.html
- nginx HTTP/2 module: https://nginx.org/en/docs/http/ngx_http_v2_module.html
- Let's Encrypt rate limits: https://letsencrypt.org/docs/rate-limits/
- Let's Encrypt challenge types: https://letsencrypt.org/docs/challenge-types/
- Let's Encrypt staging environment: https://letsencrypt.org/docs/staging-environment/
- certbot user guide and CLI reference: https://eff-certbot.readthedocs.io/
- certbot-dns-duckdns: https://pypi.org/project/certbot-dns-duckdns/
- Node.js release schedule: https://github.com/nodejs/Release/blob/main/schedule.json
- NodeSource installation notes: https://github.com/nodesource/distributions
- systemd: https://www.freedesktop.org/software/systemd/man/latest/systemd.kill.html and https://www.freedesktop.org/software/systemd/man/latest/systemd.exec.html
- OpenSSH `sshd_config`: https://man.openbsd.org/sshd_config
- Ubuntu automatic updates: https://documentation.ubuntu.com/server/how-to/software/automatic-updates/
- VirtualBox manual: https://www.virtualbox.org/manual/